When you unit-test controllers, you're testing them separately from the MVC infrastructure, such as formatting, model binding, routing, and authentication. This is obviously by design, but as with testing middleware in section 36.1, it can make testing some aspects of your controller somewhat complex.

Consider model validation. As you saw in chapter 6, one of the key responsibilities of action methods and Razor Page handlers is to check the `ModelState.IsValid` property and act accordingly if a binding model is invalid. Testing that your controllers and `PageModel`s handle validation failures correctly seems like a good candidate for a unit test.

Unfortunately, things aren't simple here either. The Razor Page/MVC framework automatically sets the `ModelState` property as part of the model-binding process. In practice, when your action method or page handler is invoked in your running app, you know that the `ModelState` will match the binding model values. But in a unit test, there's no model binding, so you must set the `ModelState` yourself manually.

Imagine you're interested in testing the error path for the controller in listing 36.4, where the model is invalid and the controller should return `BadRequestObjectResult`. In a unit test, you can't rely on the `ModelState` property being correct for the binding model. Instead, you must add a model-binding error to the controller's `ModelState` manually before calling the action, as shown in the following listing.

Listing 36.6 Testing handling of validation errors in MVC controllers

```
[Fact]
public void Convert_ReturnsBadRequestWhenInvalid()
{
    var controller = new CurrencyController();                ❶
    var model = new ConvertInputModel                         ❷
```